

DD: FUL-02-STEP-VALIDATE — Order Validation

Developer implementation guide: DD_FUL-02-STEP-VALIDATE-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_FUL-02-STEP-VALIDATE-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md The second step in the order journey. STEP-CAPTURE has just written the row and started the workflow; now FUL-02 needs to confirm the order is **actually possible**: is the HomePass sellable, is the Package active, do the technologies line up, is this customer eligible, is this not a duplicate? If yes, the journey continues to STEP-SUBSCRIPTION. If no, the order is terminally rejected and the customer is notified. Phases: IN_VALIDATION → optionally REJECTED.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/06_fulfillment/DD_FUL-02-STEP-VALIDATE-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/customers/cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
- GET /api/customers/{id}
- GET /api/homepasses/hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P
- GET /api/homepasses/{id}
- GET /api/packages/pkg_INET_VOIP_RES
- GET /api/packages/{ref}
- GET /api/subscriptions?...

- GET /api/subscriptions?customerId=cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6&homepassId=hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P&packageRef=pkg_INET_VOIP_RES&status=ACTIVE
- POST /engine-rest/external-task/ext_t_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/complete
- POST /engine-rest/external-task/ext_t_01HZ_FAIL/complete
- POST /engine-rest/external-task/fetchAndLock

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- create-subscription
- main-start
- mark-rejected
- mark-rejected-end
- order-mark-rejected
- order-validate
- validate-order

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.

8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md

The second step in the order journey. STEP-CAPTURE has just written the row and started the workflow; now FUL-02 needs to confirm the order is **actually possible**: is the HomePass sellable, is the Package active, do the technologies line up, is this customer eligible, is this not a duplicate? If yes, the journey continues to STEP-SUBSCRIPTION. If no, the order is terminally rejected and the customer is notified. Phases: IN_VALIDATION → optionally REJECTED.

1. What this step does

When STEP-CAPTURE finished, Camunda advanced the BPMN to a step called `validate-order`. This step is what Camunda calls an **external task** — a step in the recipe where Camunda doesn't do the work itself but instead puts a job on a queue called a "topic" (here, the topic is `order-validate`) and waits for our code to pick it up. The piece of our code that picks up and processes such jobs is called a **worker** — it's a Java bean inside our module that long-polls Camunda asking "anything on the `order-validate` topic for me?" When Camunda hands it a task, the worker runs, reports the result back, Camunda then advances the BPMN to whatever comes next.

The work `order-validate` does is a sequence of five checks against catalog DDs that already exist:

1. **HOME_PASS_SELLABLE** — ask RLM-CFG-01 if the HomePass is in a sellable status. RLM-CFG-01's catalog of HomePass statuses carries a `is_sellable` semantic flag per code; the actual code (RFS, LIVE, etc.) varies by deployment. We trust whatever RLM-CFG-01 says.
2. **PACKAGE_ACTIVE** — ask SIP-01 if the Package's status = `ACTIVE`. Packages in `DRAFT`, `INACTIVE`, or `END_OF_LIFE` can't be ordered.
3. **TECH_REGIONS_INTERSECT** — the Package's `target_tech_regions` (list of region codes where this Package is sold) must overlap with the HomePass's `tech_regions` (list of region codes the HomePass falls under). If they're disjoint, the Package isn't sold in the HomePass's area.
4. **NO_DUPLICATE_SUBSCRIPTION** — ask SUB-LM-01 whether this customer already has an active subscription at this HomePass for this Package. If yes, we can't create a duplicate.
5. **CUSTOMER_ELIGIBLE** — ask the customer service (v1.0: `customer_stub`; future: ILM-CFG-01) whether the customer is in a state that allows new subscriptions (KYC not blocked, no fraud flag, no eligibility hold).

If all five pass, the worker reports success and Camunda advances to STEP-SUBSCRIPTION. If any fail, the worker reports the failure detail; the BPMN's gateway after `validate-order` routes the flow to `order-mark-rejected` — a second

worker that's the terminal-failure handler for this step.

What happens on failure

Failure means *the order genuinely cannot proceed* — not a transient network blip. A duplicate subscription means the customer already has this Package at this HomePass; resubmitting won't change that. A non-sellable HomePass means we don't sell at this address yet; resubmitting won't change that either. The terminal action is:

1. The worker reports `validationOutcome = FAIL` with `validationDetails` (which check failed, why).
2. The BPMN gateway routes to `order-mark-rejected`.
3. `order-mark-rejected` sets `current_phase = REJECTED`, writes `failure_reason + failure_detail` onto the row, emits `OrderRejected` on Kafka.
4. DD_NOT-01 sees `OrderRejected` and sends the customer a notification with the failure reason in customer-friendly language ("Sorry, we don't yet service your area").
5. The Camunda process instance ends. No retry path. If the customer's circumstances change (e.g., HomePass becomes sellable later), they need to submit a new order.

What happens on upstream system unreachable

This is different from a check failing. If RLM-CFG-01, SIP-01, SUB-LM-01, or the customer service can't be reached (network error, 5xx response, timeout), the worker reports a different kind of failure to Camunda: `/failure` with `errorMessage = UPSTREAM_UNREACHABLE`. This raises a Camunda **incident** — a flag on the process instance saying "something's wrong, ops should look." The order is parked until ops resolves; the BPMN doesn't advance to `order-mark-rejected` because the order isn't actually invalid, we just couldn't check. No customer notification fires.

2. Worked example end-to-end (WIK, all checks pass)

Continuing the same scenario from STEP-CAPTURE §2: order `ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`, captured by agent `_01HZX9K8M7Q2N4P6R3T5W8Y0Z1` at 09:00 EAT, package `pkg_INET_VOIP_RES`, homepass `hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P` in Karen.

Step 1 — Camunda offers the external task on topic `order-validate`

Right after STEP-CAPTURE returned 201, Camunda advanced the BPMN from `main-start` to `validate-order`. The BPMN node for this step is an external task with topic `order-validate`. Camunda is now offering this task to anyone subscribed.

FUL-02's worker pool is long-polling Camunda. The Java bean `ValidateOrderWorker` (one of 16 such bean classes, one per topic) has a polling loop calling Camunda's fetch endpoint:

```
POST /engine-rest/external-task/fetchAndLock HTTP/1.1
Host: camunda.sophix.internal
Content-Type: application/json

{
  "workerId": "fulfillment-order-capture-worker-3",
  "asyncResponseTimeout": 30000,
  "maxTasks": 10,
  "topics": [
    { "topicName": "order-validate", "lockDuration": 5000 }
  ]
}
```

Camunda responds (the response is the worker's "you have a job" handover):

```
[
  {
    "id": "ext_t_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
    "topicName": "order-validate",
    "processInstanceId": "cam_pi_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "businessKey": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "variables": {
      "orderId": { "value": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1", "type": "String" },
      "customerId": { "value": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6", "type": "String" },
      "operatorCode": { "value": "WIK", "type": "String" },
      "packageRef": { "value": "pkg_INET_VOIP_RES", "type": "String" },
      "homepassId": { "value": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P", "type": "String" }
    },
    "lockExpirationTime": "2026-05-18T09:00:00.555Z"
  }
]
```

```
}  
]
```

The worker now has 5 seconds (the lock) to run the 5 checks and report back. If it doesn't finish in time, Camunda releases the lock and another worker can pick the task up — the worker is responsible for making its work idempotent (here, idempotency is trivial: the checks are pure reads).

Step 2 — Worker runs check **HOMEPASS_SELLABLE**

```
GET /api/homepasses/hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P HTTP/1.1  
Host: rlm-cfg-01.sophix.internal  
Authorization: Bearer <module-svc-creds>
```

RLM-CFG-01 responds:

```
{  
  "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",  
  "addressLine": "Karen Hardy, Karen, Nairobi",  
  "accessTechnology": "GPON",  
  "servicesSupported": ["DATA", "VOICE"],  
  "techRegions": ["NRB-KAREN", "NRB-LANGATA"],  
  "status": "RFS",  
  "statusFlags": {  
    "isInitial": false,  
    "isSellable": true,  
    "isActive": false,  
    "isTerminal": false  
  },  
  "networkPath": [  
    { "nodeRef": "olt_NRB_KAREN_01", "port": "GPON-3-7" },  
    { "nodeRef": "splitter_KAREN_15", "port": "P4" }  
  ]  
}
```

The worker reads `statusFlags.isSellable = true`. Check **HOMEPASS_SELLABLE** → **PASS**.

Step 3 — Worker runs check **PACKAGE_ACTIVE**

```
GET /api/packages/pkg_INET_VOIP_RES HTTP/1.1  
Host: sip-01.sophix.internal  
Authorization: Bearer <module-svc-creds>
```

SIP-01 responds (abridged):

```
{  
  "packageRef": "pkg_INET_VOIP_RES",  
  "code": "INET_VOIP_RES",  
  "name": "Residential Internet + Voice",  
  "status": "ACTIVE",  
  "currentVersionId": "pkv_01HZ...",  
  "targetTechRegions": ["NRB-KAREN", "NRB-LANGATA", "NRB-WESTLANDS", "MSA-NYALI"],  
  "serviceRefs": ["svc_INET_GPON_100M", "svc_VOIP_HOSTED"],  
  "billingFrequency": "MONTHLY",  
  "priceMinor": 1500000,  
  "currency": "KES"  
}
```

The worker reads `status = ACTIVE`. Check **PACKAGE_ACTIVE** → **PASS**.

Step 4 — Worker runs check **TECH_REGIONS_INTERSECT**

The worker intersects the two arrays it already has from steps 2 and 3:

- HomePass tech regions: ["NRB-KAREN", "NRB-LANGATA"]
- Package target tech regions: ["NRB-KAREN", "NRB-LANGATA", "NRB-WESTLANDS", "MSA-NYALI"]

Intersection: ["NRB-KAREN", "NRB-LANGATA"] — non-empty. Check **TECH_REGIONS_INTERSECT** → **PASS** with detail "NRB-KAREN, NRB-LANGATA".

No external call is made for this check; it's computed locally from data already fetched.

Step 5 — Worker runs check **NO_DUPLICATE_SUBSCRIPTION**

```
GET /api/subscriptions?customerId=cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6&homepassId=hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P&packageRef=pkg_INET_VOIP_RES HTTP/1.1  
Host: sub-lm-01.sophix.internal  
Authorization: Bearer <module-svc-creds>
```

SUB-LM-01 responds:

```
{
  "subscriptions": [],
  "totalCount": 0
}
```

No existing active subscription matches. Check **NO_DUPLICATE_SUBSCRIPTION** → **PASS**.

Step 6 — Worker runs check CUSTOMER_ELIGIBLE

```
GET /api/customers/cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6 HTTP/1.1
Host: fulfillment-order-capture.sophix.internal/customer-stub-api
Authorization: Bearer <module-svc-creds>
```

(In v1.0 this is FUL-02's own customer_stub admin endpoint; post-ILM-CFG-01 it routes to ILM-CFG-01.)

```
{
  "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "name": "John Mwangi",
  "msisdn": "+254712345678",
  "email": "john.mwangi@example.com",
  "kycStatus": "PENDING",
  "eligibility": "ELIGIBLE",
  "operatorCode": "WIK"
}
```

eligibility = ELIGIBLE. (Note: KYC at this point is PENDING — that's fine; in WIK the KYC two-level approval happens later in STEP-KYC, after payment. The eligibility flag is a separate concept: it's a customer-level block for fraud, regulatory holds, etc., set by ops manually.) Check **CUSTOMER_ELIGIBLE** → **PASS**.

Step 7 — Worker reports success to Camunda

All 5 checks passed. The worker updates the customer_order row with the validation outcomes:

```
UPDATE customer_order
SET validation_outcomes = '[
  {"check": "HOMEPASS_SELLABLE", "result": "PASS"},
  {"check": "PACKAGE_ACTIVE", "result": "PASS"},
  {"check": "TECH_REGIONS_INTERSECT", "result": "PASS", "detail": "NRB-KAREN, NRB-LANGATA"},
  {"check": "NO_DUPLICATE_SUBSCRIPTION", "result": "PASS"},
  {"check": "CUSTOMER_ELIGIBLE", "result": "PASS"}
]':::jsonb,
last_updated_at = NOW()
WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

Then it tells Camunda the task completed successfully:

```
POST /engine-rest/external-task/ext_t_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/complete HTTP/1.1
Host: camunda.sophix.internal
Content-Type: application/json
```

```
{
  "workerId": "fulfillment-order-capture-worker-3",
  "variables": {
    "validationOutcome": { "value": "PASS",
    "validationDetails": { "value": "[{"check": "HOMEPASS_SELLABLE", "result": "PASS"}, {"check": "PACKAGE_ACTIVE",
  }
}
```

Camunda 204. The BPMN advances past validate-order through the success-branch gateway to create-subscription (STEP-SUBSCRIPTION's external task).

Step 8 — Worker emits OrderValidated on Kafka

The worker also publishes:

```
{
  "eventType": "OrderValidated",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T09:00:01.234Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "packageRef": "pkg_INET_V0IP_RES",
    "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",

    "validationDetails": [
      { "check": "HOMEPASS_SELLABLE", "result": "PASS" },
      { "check": "PACKAGE_ACTIVE", "result": "PASS" },
      { "check": "TECH_REGIONS_INTERSECT", "result": "PASS", "detail": "NRB-KAREN, NRB-LANGATA" },
    ]
  }
}
```

```

    { "check": "NO_DUPLICATE_SUBSCRIPTION", "result": "PASS" },
    { "check": "CUSTOMER_ELIGIBLE", "result": "PASS" }
  ]
}
}

```

Total elapsed inside this step: ~600 ms (mostly waiting on the four GETs).

3. Worked example — what a failure looks like

For contrast: order `ord_01HZ...D` captured by a backoffice user for a customer trying to order package `pkg_TV_HFC_BASIC` (HFC-only TV package) at a HomePass `hp_01HX_KAREN_99` that only has GPON tech regions.

The worker runs:

- Check `HOMEPASS_SELLABLE`: HomePass status flag `isSellable = true` → PASS.
- Check `PACKAGE_ACTIVE`: Package status = ACTIVE → PASS.
- Check `TECH_REGIONS_INTERSECT`: HomePass tech regions `["NRB-KAREN"]`; Package target tech regions `["MSA-OLD-TOWN", "MSA-NYALI"]`. Intersection: empty. **FAIL** with detail `[] = [NRB-KAREN] ∩ [MSA-OLD-TOWN, MSA-NYALI]`.

The worker stops here (no need to run further checks once one fails) and reports:

```
POST /engine-rest/external-task/ext_t_01HZ_FAIL/complete HTTP/1.1
```

```

{
  "workerId": "fulfillment-order-capture-worker-3",
  "variables": {
    "validationOutcome": { "value": "FAIL", "type": "String" },
    "validationDetails": { "value": "[{"check\":\"HOMEPASS_SELLABLE\", \"result\":\"PASS\"}, {"check\":\"PACKAGE_ACTIVE\", \"result\":\"PASS\"}], {"check\":\"TECH_REGIONS_INTERSECT\", \"result\":\"FAIL\", \"detail\":\"[] = [NRB-KAREN] ∩ [MSA-OLD-TOWN, MSA-NYALI]\"}]", "type": "String" },
    "failureReason": { "value": "PACKAGE_NOT_AVAILABLE_IN_HOMEPASS_REGION", "type": "String" },
    "failureDetail": { "value": "Package pkg_TV_HFC_BASIC requires tech regions [MSA-OLD-TOWN, MSA-NYALI]; HomePass hp_01HX_KAREN_99", "type": "String" }
  }
}

```

The BPMN gateway sees `validationOutcome = FAIL` and routes to topic `order-mark-rejected`. The `MarkOrderRejectedWorker` (another Java bean in our module) picks up the next task, writes:

```

UPDATE customer_order
SET current_phase      = 'REJECTED',
    failure_reason      = 'PACKAGE_NOT_AVAILABLE_IN_HOMEPASS_REGION',
    failure_detail      = 'Package pkg_TV_HFC_BASIC requires tech regions [MSA-OLD-TOWN, MSA-NYALI]; HomePass hp_01HX_KAREN_99',
    last_updated_at     = NOW()
WHERE order_id          = 'ord_01HZ...D';

```

and emits `OrderRejected` on Kafka:

```

{
  "eventType": "OrderRejected",
  "aggregateId": "ord_01HZ...D",
  "timestamp": "2026-05-18T09:01:00.789Z",
  "payload": {
    "orderId": "ord_01HZ...D",
    "customerId": "cust_01HX...D",
    "operatorCode": "WIK",
    "packageRef": "pkg_TV_HFC_BASIC",
    "homepassId": "hp_01HX_KAREN_99",
    "failureReason": "PACKAGE_NOT_AVAILABLE_IN_HOMEPASS_REGION",
    "failureDetail": "Package pkg_TV_HFC_BASIC requires tech regions [MSA-OLD-TOWN, MSA-NYALI]; HomePass hp_01HX_KAREN_99",
    "validationDetails": [
      { "check": "HOMEPASS_SELLABLE", "result": "PASS" },
      { "check": "PACKAGE_ACTIVE", "result": "PASS" },
      { "check": "TECH_REGIONS_INTERSECT", "result": "FAIL", "detail": "[] = [NRB-KAREN] ∩ [MSA-OLD-TOWN, MSA-NYALI]" }
    ],
    "rejectedAt": "2026-05-18T09:01:00.789Z"
  }
}

```

`DD_NOT-01` consumes the event and SMSes the customer: "Sorry, the TV package isn't available in your area yet — please contact support for alternatives." Camunda closes the process instance. Total elapsed: ~400 ms (the worker bailed out early on the third check).

4. Workers

4.1 order-validate

Field	Value
Topic	order-validate
Purpose	Run the 5-check matrix on a captured order; return PASS or FAIL with detail.
Process variables read	orderId, customerId, packageRef, homepassId, operatorCode
External HTTP calls	RLM-CFG-01 GET /api/homepasses/{id} (HomePass sellable + tech regions). SIP-01 GET /api/packages/{ref} (Package status + target tech regions). SUB-LM-01 GET /api/subscriptions?... (duplicate check). Customer service GET /api/customers/{id} (eligibility).
Process variables written	validationOutcome (PASS / FAIL), validationDetails (JSON array of {check, result, detail}), and on FAIL also failureReason + failureDetail for the next worker
customer_order columns written	validation_outcomes
Kafka emissions	On PASS: sophix.orders.validated (this worker). On FAIL: nothing here — emission is order-mark-rejected's job.
Lock duration	5000 ms
Retry policy	0 retries on /failure. Validation should be deterministic; non-deterministic failure is a bug, not retry-worthy.

Check matrix:

Check	What it does
HOMEPASS_SELLABLE	Read RLM-CFG-01; verify statusFlags.isSellable = true
PACKAGE_ACTIVE	Read SIP-01; verify status = ACTIVE
TECH_REGIONS_INTERSECT	Compute intersection of HomePass techRegions $\cap$ Package targetTechRegions; must be non-empty
NO_DUPLICATE_SUBSCRIPTION	Read SUB-LM-01 filtered on (customerId, homepassId, packageRef, status=ACTIVE); verify result is empty
CUSTOMER_ELIGIBLE	Read customer service; verify eligibility = ELIGIBLE (not BLOCKED, FRAUD_HOLD, REGULATORY_HOLD)

The worker short-circuits on first failure (doesn't run the remaining checks); the validationDetails array carries only the checks that ran. The order of evaluation is HomePass → Package → TechRegions intersect → Duplicate → CustomerEligible.

Failure modes:

- **A check fails** → worker completes with validationOutcome = FAIL and the detail. Camunda gateway routes to order-mark-rejected. Customer is eventually notified (via DD_NOT-01 on OrderRejected). Order is terminal.
- **Upstream system unreachable** (RLM-CFG-01 down, SIP-01 down, etc.) → worker reports /failure with errorMessage = UPSTREAM_UNREACHABLE. Camunda raises an incident; ops is alerted via NOC; the order is parked until ops resolves. No customer notification fires.

4.2 order-mark-rejected

Field	Value
Topic	order-mark-rejected
Purpose	Terminal failure handler. Marks the order as REJECTED, persists the failure detail, emits the rejection Kafka event.
Process variables read	orderId, operatorCode, failureReason, failureDetail, validationDetails
External HTTP calls	None — pure DB write + Kafka emit.
Process variables written	None — this is a terminal worker; the BPMN ends after it.
customer_order columns written	current_phase = REJECTED, failure_reason, failure_detail
Kafka emissions	sophix.orders.rejected
Lock duration	5000 ms
Retry policy	3 retries with PT30S between attempts (Kafka transient errors).

5. DB tables touched

customer_order (FRAMEWORK §5.1) — UPDATE on the existing row

STEP-VALIDATE doesn't INSERT; it updates the row written by STEP-CAPTURE. Columns set by this step:

Column	When set	Set by	Example value (PASS)	Example value (FAIL)
current_phase	On Camunda entry to validate-order	PhaseProjector	IN_VALIDATION	IN_VALIDATION → REJECTED
current_activity_id	On Camunda transition	PhaseProjector	validate-order → create-subscription	validate-order → mark-rejected → end
validation_outcomes	After all checks run	order-validate worker	[{"check": "HOMEPASS_SELLABLE", "result": "PASS"}, ...] (5 entries on PASS)	[{"check": "HOMEPASS_SELLABLE", "result": "PASS"}, {"check": "PACKAGE_ACTIVE", "result": "PASS"}, {"check": "TECH_REGIONS_INTERSECT", "result": "FAIL", "detail": "..."}] (truncated at the failing check)
failure_reason	On FAIL only	order-mark-rejected worker	NULL	One of: HOMEPASS_NOT_SELLABLE, PACKAGE_NOT_ACTIVE, PACKAGE_NOT_AVAILABLE_IN_HOMEPASS_REGION, DUPLICATE_SUBSCRIPTION, CUSTOMER_NOT_ELIGIBLE
failure_detail	On FAIL only	order-mark-rejected worker	NULL	Free-text describing what failed, e.g. Package pkg_TV_HFC_BASIC requires tech regions [MSA-OLD-TOWN, MSA-NYALI]; HomePass hp_01HX_KAREN_99 is in [NRB-KAREN]
last_updated_at	On any UPDATE	trigger / worker	2026-05-18 09:00:01.234	2026-05-18 09:01:00.789

Sample data — five orders post-validation, mixing PASS and FAIL outcomes:

order_id	operator_code	package_ref	homepass_id	current_phase	validation_outcomes (truncated)	failure_reason
ord_01HZ...A	WIK	pkg_INET_VOIP_RES	hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P	IN_VALIDATION (advancing to STEP-SUBSCRIPTION)	5×PASS	NULL
ord_01HZ...B	WIK	pkg_INET_100M_ONLY	hp_01HX_LAVINGTON_42	IN_VALIDATION (advancing to STEP-SUBSCRIPTION)	5×PASS	NULL
ord_01HZ...D	WIK	pkg_TV_HFC_BASIC	hp_01HX_KAREN_99	REJECTED	2×PASS + TECH_REGIONS_INTERSECT FAIL	PACKAGE_NOT_AVAILABLE_IN_HOMEPASS_REGION
ord_01HZ...G	WIK	pkg_INET_VOIP_RES	hp_01HX_DRAFTING_07	REJECTED	HOMEPASS_SELLABLE FAIL	HOMEPASS_NOT_SELLABLE
ord_01HZ...H	WIK	pkg_INET_500M_PRO	hp_01HX_WESTLANDS_05	IN_VALIDATION (advancing to STEP-SUBSCRIPTION)	5×PASS	NULL

customer_order_phase_history (FRAMEWORK §5.2) — one or two rows appended

On PASS — one row, transitioning IN_VALIDATION → next step's phase:

Column	Example value
id	ordh_01HZX9K8M7Q2N4P6R3T5W8Y0Z3
order_id	ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
previous_phase	IN_VALIDATION
new_phase	IN_VALIDATION (subscription creation is transitive — no dedicated phase)
previous_activity_id	validate-order

Column	Example value
new_activity_id	create-subscription
transition_at	2026-05-18 09:00:01.234
triggered_by	worker_completion
triggered_by_detail	ValidateOrderWorker
metadata	{"validationOutcome":"PASS","checksRun":5}

On FAIL — two rows, one for each Camunda transition:

id	previous_phase	new_phase	new_activity_id	triggered_by	metadata
ordh_01HZ...D_1	IN_VALIDATION	IN_VALIDATION	mark-rejected	worker_completion	{"validationOutcome":"FAIL","failedCheck":"TECH_REGIONS_INTERSECT"}
ordh_01HZ...D_2	IN_VALIDATION	REJECTED	mark-rejected-end	worker_completion	{"failureReason":"PACKAGE_NOT_AVAILABLE_IN_HOMEPASS_REGION"}

6. Kafka events emitted

sophix.orders.validated

Emitted by order-validate after all checks pass.

```
{
  "eventType": "OrderValidated",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T09:00:01.234Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "packageRef": "pkg_INET_V0IP_RES",
    "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",

    "validationDetails": [
      { "check": "HOMEPASS_SELLABLE", "result": "PASS" },
      { "check": "PACKAGE_ACTIVE", "result": "PASS" },
      { "check": "TECH_REGIONS_INTERSECT", "result": "PASS", "detail": "NRB-KAREN, NRB-LANGATA" },
      { "check": "NO_DUPLICATE_SUBSCRIPTION", "result": "PASS" },
      { "check": "CUSTOMER_ELIGIBLE", "result": "PASS" }
    ]
  }
}
```

Consumers and what they do with it:

Consumer	Purpose
CVM	Funnel stage: counts orders that survived validation. Drop-off here means catalog/inventory config issues.
NOC	Sanity counter — validation should pass at ~99%+ rates; drops below this trigger alerts.
Finance ETL	Validated orders are higher-probability activations than just-captured ones; forecast model updates.

DD_NOT-01 does NOT consume OrderValidated — at this point the customer doesn't need to hear anything; the next notification is at STEP-PAYMENT when the customer needs to pay.

sophix.orders.rejected

Emitted by order-mark-rejected (NOT by order-validate).

```
{
  "eventType": "OrderRejected",
  "aggregateId": "ord_01HZ...D",
  "timestamp": "2026-05-18T09:01:00.789Z",
  "payload": {
    "orderId": "ord_01HZ...D",
```

```

    "customerId": "cust_01HX...D",
    "operatorCode": "WIK",
    "packageRef": "pkg_TV_HFC_BASIC",
    "homepassId": "hp_01HX_KAREN_99",
    "failureReason": "PACKAGE_NOT_AVAILABLE_IN_HOMEPASS_REGION",
    "failureDetail": "Package pkg_TV_HFC_BASIC requires tech regions [MSA-OLD-TOWN, MSA-NYALI]; HomePass hp_01HX_KAREN_99",
    "validationDetails": [
      { "check": "HOMEPASS_SELLABLE", "result": "PASS" },
      { "check": "PACKAGE_ACTIVE", "result": "PASS" },
      { "check": "TECH_REGIONS_INTERSECT", "result": "FAIL", "detail": "[ ] = [NRB-KAREN] ∩ [MSA-OLD-TOWN, MSA-NYALI]" }
    ],
    "rejectedAt": "2026-05-18T09:01:00.789Z"
  }
}

```

Consumers:

Consumer	Purpose
DD_NOT-01	Customer notification with the failure reason in customer-friendly language ("Sorry, your area isn't covered for this Package yet"). The only consumer that actually contacts the customer for a rejection.
CVM	Funnel rejection tracking — which checks fail most often, in which regions, for which packages. Drives catalog/coverage improvement decisions.
NOC	Rejection-rate dashboards. A sudden spike in HOMEPASS_NOT_SELLABLE rejections might mean an RLM-CFG-01 catalog regression.

7. Step-pinned rules

ID	Rule
R-FUL-02-VAL-1	order-validate runs the 5-check matrix on every captured order. Checks evaluate in order HomePass → Package → TechRegions intersect → Duplicate → CustomerEligible; the worker short-circuits on first failure.
R-FUL-02-VAL-2	The check sequence is deterministic and operator-agnostic — workers don't depend on operator shape. The KE WIK BPMN runs the standard 5-check sequence. Other operator BPMNs may parameterize the check set (e.g., skip duplicate check when subscription is deferred) via future configuration; v1.0 ships only the WIK BPMN.
R-FUL-02-VAL-3	order-validate has 0 retries on /failure. Validation should be deterministic on retry; a non-deterministic upstream call is reported as UPSTREAM_UNREACHABLE and raises a Camunda incident, parking the order until ops resolves (no auto-rejection, no customer notification — the order is genuinely fine, we just couldn't check).
R-FUL-02-VAL-4	Worker short-circuits on first FAIL — doesn't run remaining checks. The validationDetails array carries only the checks that ran. Order of evaluation: HomePass → Package → TechRegions intersect → Duplicate → CustomerEligible.
R-FUL-02-VAL-5	Any check FAIL → validationOutcome = FAIL with failureReason (enum) + failureDetail (free-text). BPMN gateway routes to order-mark-rejected.
R-FUL-02-VAL-6	order-mark-rejected is terminal: phase REJECTED, no recovery path. If circumstances change (HomePass becomes sellable, customer ineligibility lifted), the customer must submit a new order — the existing rejected order is not re-validated.
R-FUL-02-VAL-7	The OrderRejected Kafka event is emitted by order-mark-rejected, NOT by order-validate. The split is deliberate: the validate worker returns a verdict; the mark-rejected worker turns that verdict into the visible terminal state + event. DD_NOT-01 only consumes OrderRejected.
R-FUL-02-VAL-8	RLM-CFG-01 / SIP-01 lookups are cacheable (Package + HomePass change rates are low). Cache TTL 5 minutes inside the worker JVM. SUB-LM-01 + customer-service lookups are NOT cached (customer state can change at any moment).
R-FUL-02-VAL-9	failureReason enum is closed: HOMEPASS_NOT_SELLABLE, PACKAGE_NOT_ACTIVE, PACKAGE_NOT_AVAILABLE_IN_HOMEPASS_REGION (tech regions disjoint), DUPLICATE_SUBSCRIPTION, CUSTOMER_NOT_ELIGIBLE. Adding a new failure category requires extending DD_NOT-01's customer-message templates first.